

FINAL YEAR PROJECT – COMPLETE SETUP GUIDE

UPDATED VERSION – Reflects Actual Working Configuration

From Zero to

Autonomous Robot Navigation

Windows + MobaXterm | Ubuntu 22.04 | Raspberry Pi 4
QBot 2 | Kinect v1 | ROS 2 Humble | SLAM Toolbox | Nav2

Student	Baanujan E/20/030
Supervisor	Dr. D.H.S. Maithripala
Hardware	Raspberry Pi 4 + Quanser QBot 2 + Xbox Kinect v1
Software	Ubuntu 22.04 LTS + ROS 2 Humble + SLAM Toolbox + Nav2
Development PC	Windows with MobaXterm (SSH + X11 forwarding)
Pi Username	kobuki
Goal	SLAM map building + autonomous navigation

This guide reflects the actual commands that worked on this specific hardware.

Changed sections are marked with UPDATED boxes.

Contents

How to Use This Guide	3
Download Raspberry Pi Imager	4
Check MobaXterm X11 Settings	4
Connect from MobaXterm	6
Step 1 – Set Locale	8
Step 2 – Add ROS 2 Repository	8
Step 3 – Install ROS 2 Humble	8
Step 4 – Configure Environment	9
Step 5 – Test ROS 2	9
Create Workspace	10
Download Kobuki Packages	10
Install Dependencies and Build	10
Connect and Test QBot 2	10
Step 1 – Install libfreenect	12
Step 2 – Test Kinect Hardware	12
Step 3 – Clone KinectV1-Ros2 Package	12
Step 4 – Build (skip mic package)	13
Step 5 – Load Kinect Kernel Module Fix	13
Step 6 – Test Kinect in ROS 2	13
Step 1 – Install Package	15
Step 2 – Create Camera Info Fixer Script	15
Step 3 – Create depth_to_scan Launch File	16
Step 4 – Create package.xml	17
Step 5 – Create CMakeLists.txt	17
Step 6 – Build	17
Step 7 – Test the Full Chain	18
Install SLAM Toolbox	22
Launch Everything for Mapping	22
Tab 1 – Kobuki Driver	22
Tab 2 – Kinect Depth Node	22

Tab 3 – Camera Info Fixer	22
Tab 4 – Depth to LaserScan	22
Tab 5 – Robot State Publisher	23
Tab 6 – SLAM Toolbox	23
Verify All Topics are Running	23
Drive the Robot to Build the Map	23
Watch the Map in RViz2	24
Save the Map	25
Install Nav2	26
Create Nav2 Parameters File	26
Launch Full Navigation Stack	27
Send Navigation Goals with RViz2	28
Troubleshooting	29
Quick Reference Commands	30
14-Day Practical Plan	31

How to Use This Guide

This is the updated version of the setup guide. It reflects the actual commands that worked during real hardware testing on this Raspberry Pi with username `kobuki`.

Symbol	Meaning
NOTE	Important information.
WARNING	Skip this and you will get an error.
TIP	Helpful advice.
WINDOWS	Run on your Windows laptop, not the Pi.
CONFIRMED WORKING	This exact command was tested and worked.
UPDATED	This section differs from the original guide.

UPDATED – Different from original guide

Key differences from original guide:

- Username is `kobuki`, not `ubuntu`
- Phase 3: Fixed typo in `dpkg -print-architecture` command
- Phase 5: OpenNI2 does not work with Kinect v1. Replaced with `KinectV1-Ros2` package
- Kinect topics are `/kinect/depth/image_raw` not `/camera/depth/image_raw`
- Phase 6: Remappings updated to kinect topics. Added camera info fixer script.
- Phase 9: SLAM Toolbox used instead of RTAB-Map. Kobuki `cmd_vel` remapped to `/commands/velocity`

PHASE 0

Set Up Your Windows Laptop

Download Raspberry Pi Imager

WINDOWS

Go to <https://www.raspberrypi.com/software/> and download Raspberry Pi Imager for Windows.

Check MobaXterm X11 Settings

WINDOWS

Open MobaXterm → Settings → Configuration → X11 tab. Make sure X11 forwarding is enabled. Click OK.

PHASE 1

Flash Ubuntu 22.04 onto Raspberry Pi 4

WARNING

This erases everything on the SD card. The current system used username `kobuki`. After flashing Ubuntu, the username will be `ubuntu` unless you set it differently in Imager settings.

STEP 1 – Insert MicroSD card into laptop

STEP 2 – Open Raspberry Pi Imager

STEP 3 – Choose OS

Click Choose OS → Other general-purpose OS → Ubuntu → **Ubuntu Server 22.04 LTS 64-bit**. Do not choose Desktop.

STEP 4 – Choose Storage

Select your MicroSD card carefully.

STEP 5 – Open Advanced Settings (gear icon)

Hostname:	raspberrypi
Enable SSH:	Yes
Username:	kobuki
Password:	your chosen password
WiFi SSID:	lab WiFi name
WiFi Password:	lab WiFi password
WiFi Country:	LK

NOTE

Username is set to `kobuki` to match the existing setup. All paths in this guide use `/home/kobuki/`.

STEP 6 – Write and boot

Click Write. Wait for it to finish. Insert card into Pi. Connect Ethernet. Power on. Wait 3 minutes.

Connect from MobaXterm

WINDOWS

MobaXterm → Session → SSH → Remote host: `raspberrypi.local`, Username: `kobuki`, Port: 22 → OK.

```
# You should see:  
kobuki@pi:~$
```


PHASE 2

Update Ubuntu and Install Essential Tools

```
sudo apt update  
sudo apt upgrade -y  
sudo apt install -y curl wget git build-essential python3-pip  
sudo reboot
```

PHASE 3

Install ROS 2 Humble

UPDATED – Different from original guide

The original guide had a typo in the repository setup command. The corrected commands are below.

Step 1 – Set Locale

```
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Step 2 – Add ROS 2 Repository

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install -y curl

sudo curl -sSL \
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
```

UPDATED – Different from original guide

The command below must have a space between `dpkg` and `-print-architecture`. The original guide had a typo (`dpkg-print-architecture`) which caused a “command not found” error.

```
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

CONFIRMED WORKING

Note the space in `dpkg -print-architecture`. Without this space the command fails with “command not found”.

Step 3 – Install ROS 2 Humble

```
sudo apt update
sudo apt install -y ros-humble-ros-base
sudo apt install -y python3-colcon-common-extensions \
python3-rosdep python3-argcomplete
```

Step 4 – Configure Environment

```
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

Step 5 – Test ROS 2

```
# TAB 1:
ros2 run demo_nodes_cpp talker

# TAB 2:
ros2 run demo_nodes_cpp listener
# Should print: [INFO] I heard: Hello World: 1 ...
```

PHASE 4

Install Kobuki Driver for QBot 2 Wheels

Create Workspace

```
mkdir -p ~/fyp_ws/src
cd ~/fyp_ws/src
```

Download Kobuki Packages

```
git clone https://github.com/kobuki-base/kobuki_ros.git
git clone https://github.com/kobuki-base/kobuki_ros_interfaces.git
git clone https://github.com/kobuki-base/kobuki_core.git
git clone https://github.com/kobuki-base/velocity_smoother.git
git clone https://github.com/stonier/ecl_tools.git
```

Install Dependencies and Build

```
cd ~/fyp_ws
sudo rosdep init
rosdep update
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install

echo 'source ~/fyp_ws/install/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

Connect and Test QBot 2

```
# Connect QBot 2 to Pi via USB. Then:
ls /dev/ttyUSB*
# Should show: /dev/ttyUSB0

sudo chmod 666 /dev/ttyUSB0
sudo usermod -aG dialout kobuki
sudo reboot
```

After reboot:

```
ros2 launch kobuki_ros kobuki_node-launch.py
```

NOTE

The Kobuki driver remaps velocity commands. Use `/commands/velocity` topic to move the robot, not `/cmd_vel`. This is important for teleoperation in Phase 9.

In another tab:

```
ros2 topic list
# Look for: /odom /commands/velocity /joint_states
```

PHASE 5

Install Kinect Driver

UPDATED – Different from original guide

The original guide used OpenNI2. This did not work with Kinect v1 on this system. `ros-humble-openni2-launch` package does not exist in ROS 2 Humble.

Even `openni2_camera` showed “0 devices connected” with Kinect v1.

Solution: Use the `KinectV1-Ros2` package which uses `libfreenect` directly.

Step 1 – Install libfreenect

```
sudo apt install -y libfreenect0.5 libfreenect-bin \
    libfreenect-dev freenect
```

Step 2 – Test Kinect Hardware

```
# Check Kinect is detected:
lsusb
# Look for:
# Microsoft Corp. Xbox NUI Camera
# Microsoft Corp. Xbox NUI Audio
# Microsoft Corp. Kinect for Windows NUI Motor

# Test raw streams (requires monitor or X11):
freenect-glvview
```

CONFIRMED WORKING

Kinect v1 was confirmed working with `freenect`: `lsusb` showed Xbox NUI Camera, Xbox NUI Audio, and Kinect for Windows NUI Motor. `freenect-glvview` displayed RGB and depth streams successfully.

Step 3 – Clone KinectV1-Ros2 Package

```
mkdir ~/Ros2-KinectV1
cd ~/Ros2-KinectV1
git clone -b ros2-humble \
    https://github.com/SriharshaShesham/KinectV1-Ros2.git .
```

Step 4 – Build (skip mic package)

```
cd ~/Ros2-KinectV1
source /opt/ros/humble/setup.bash
colcon build --symlink-install \
  --packages-skip ros2_kinect_mic_node
```

WARNING

`ros2_kinect_mic_node` will fail to build because of a missing header path. This is expected and does not matter. The mic is not needed for SLAM or navigation. Skip it with `-packages-skip`.

```
# Source the workspace (ignore the mic warning):
source ~/Ros2-KinectV1/install/setup.bash

# Add to bashrc:
echo 'source ~/Ros2-KinectV1/install/setup.bash' >> ~/.bashrc
```

Step 5 – Load Kinect Kernel Module Fix

UPDATED – Different from original guide

Before running any Kinect ROS 2 node, you must unload the conflicting kernel module. Otherwise you get `LIBUSB_ERROR_BUSY`.

```
sudo modprobe -r gspca_kinect
```

TIP

Run this command every time before starting the Kinect node after a reboot. To make it permanent, add it to `/etc/rc.local` or create a udev rule.

Step 6 – Test Kinect in ROS 2

```
# TAB 1 -- Run depth node:
cd ~/Ros2-KinectV1
source /opt/ros/humble/setup.bash
source install/setup.bash
sudo modprobe -r gspca_kinect
ros2 run ros2_kinect_depth depth_node

# TAB 2 -- Check topics:
source /opt/ros/humble/setup.bash
source ~/Ros2-KinectV1/install/setup.bash
```

```
ros2 topic list
```

CONFIRMED WORKING

Confirmed working. The following topics were published:

```
/kinect/depth/camera_info
```

```
/kinect/depth/image_raw
```

```
# Verify data rate:
```

```
ros2 topic hz /kinect/depth/image_raw
```

```
# Expected: average rate: 30.000
```

WARNING

Only one node can access the Kinect USB device at a time. Do not run `depth_node` and `rgb_node` simultaneously from separate terminals – they will conflict with `LIBUSB_ERROR_BUSY`. Run `depth_node` only for SLAM (depth is what matters).

PHASE 6

Convert Kinect Depth to Laser Scan

UPDATED – Different from original guide

Topic names are different from the original guide. Use `/kinect/depth/image_raw` and `/kinect/depth/camera_info`, not `/camera/depth/image_raw`. The output frame is `kinect_depth_frame`. Also a camera info fixer script is needed because the Kinect node does not publish full `camera_info` by default.

Step 1 – Install Package

```
sudo apt install -y ros-humble-depthimage-to-laserscan
```

Step 2 – Create Camera Info Fixer Script

UPDATED – Different from original guide

The `depthimage_to_laserscan` node requires valid `camera_info` data. The Kinect node publishes incomplete `camera_info`. This Python script fixes it by republishing corrected camera info.

```
nano ~/fix_kinect_camera_info.py
```

Paste this script:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import CameraInfo

class CameraInfoFixer(Node):
    def __init__(self):
        super().__init__('camera_info_fixer')
        self.sub = self.create_subscription(
            CameraInfo,
            '/kinect/depth/camera_info',
            self.callback,
            10)
        self.pub = self.create_publisher(
            CameraInfo,
            '/kinect/depth/camera_info_fixed',
            10)

    def callback(self, msg):
```

```
msg.k[0] = 570.3422241210938
msg.k[2] = 319.5
msg.k[4] = 570.3422241210938
msg.k[5] = 239.5
msg.k[8] = 1.0
msg.p[0] = 570.3422241210938
msg.p[2] = 319.5
msg.p[5] = 570.3422241210938
msg.p[6] = 239.5
msg.p[10] = 1.0
self.pub.publish(msg)

def main():
    rclpy.init()
    node = CameraInfoFixer()
    rclpy.spin(node)

if __name__ == '__main__':
    main()
```

Step 3 – Create `depth_to_scan` Launch File

```
mkdir -p ~/fyp_ws/src/qbot2_bringup/launch
nano ~/fyp_ws/src/qbot2_bringup/launch/depth_to_scan.launch.py
```

Paste:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='depthimage_to_laserscan',
            executable='depthimage_to_laserscan_node',
            name='depthimage_to_laserscan',
            remappings=[
                ('depth', '/kinect/depth/image_raw'),
                ('depth_camera_info',
                 '/kinect/depth/camera_info_fixed'),
                ('scan', '/scan'),
            ],
            parameters=[{
                'scan_height': 10,
                'range_min': 0.45,
                'range_max': 4.0,
```

```
        'output_frame': 'kinect_depth_frame',  
    }  
)  
])
```

Step 4 – Create package.xml

```
nano ~/fyp_ws/src/qbot2_bringup/package.xml
```

Paste:

```
<?xml version="1.0"?>  
<package format="3">  
  <name>qbot2_bringup</name>  
  <version>0.0.1</version>  
  <description>QBot 2 bringup launch files</description>  
  <maintainer email="test@test.com">kobuki</maintainer>  
  <license>Apache-2.0</license>  
  <buildtool_depend>ament_cmake</buildtool_depend>  
  <exec_depend>depthimage_to_laserscan</exec_depend>  
  <exec_depend>launch</exec_depend>  
  <exec_depend>launch_ros</exec_depend>  
</package>
```

Step 5 – Create CMakeLists.txt

```
nano ~/fyp_ws/src/qbot2_bringup/CMakeLists.txt
```

Paste:

```
cmake_minimum_required(VERSION 3.8)  
project(qbot2_bringup)  
find_package(ament_cmake REQUIRED)  
install(DIRECTORY launch  
  DESTINATION share/${PROJECT_NAME}/)  
ament_package()
```

Step 6 – Build

```
cd ~/fyp_ws  
source /opt/ros/humble/setup.bash  
colcon build --symlink-install \
```

```
--packages-select qbot2_bringup
source install/setup.bash
```

Step 7 – Test the Full Chain

Use 4 MobaXterm tabs simultaneously:

```
# TAB 1 -- Kinect depth node:
cd ~/Ros2-KinectV1
source /opt/ros/humble/setup.bash
source install/setup.bash
sudo modprobe -r gspca_kinect
ros2 run ros2_kinect_depth depth_node

# TAB 2 -- Camera info fixer:
source /opt/ros/humble/setup.bash
python3 ~/fix_kinect_camera_info.py

# TAB 3 -- Depth to LaserScan:
cd ~/fyp_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch qbot2_bringup depth_to_scan.launch.py

# TAB 4 -- Check /scan:
source /opt/ros/humble/setup.bash
ros2 topic list | grep scan
ros2 topic hz /scan
ros2 topic echo /scan --once
```

CONFIRMED WORKING

Expected chain:

```
Kinect depth → /kinect/depth/image_raw → camera_info_fixer →
/kinect/depth/camera_info_fixed → depthimage_to_laserscan → /scan
```

PHASE 7

Create Robot Description – URDF

WARNING

Measure the actual height of the Kinect from the floor before editing this file. If the Kinect is 28 cm high, use 0.28.

```
mkdir -p ~/fyp_ws/src/qbot2_description/urdf
nano ~/fyp_ws/src/qbot2_description/urdf/qbot2.urdf
```

Paste:

```
<?xml version="1.0"?>
<robot name="qbot2">

  <link name="base_footprint"/>

  <joint name="base_joint" type="fixed">
    <parent link="base_footprint"/>
    <child link="base_link"/>
    <origin xyz="0 0 0.0102" rpy="0 0 0"/>
  </joint>

  <link name="base_link">
    <visual><geometry>
      <cylinder length="0.10" radius="0.178"/>
    </geometry></visual>
  </link>

  <!-- Change 0.28 to actual Kinect height in metres -->
  <joint name="camera_joint" type="fixed">
    <parent link="base_link"/>
    <child link="camera_link"/>
    <origin xyz="0.05 0.0 0.28" rpy="0 0 0"/>
  </joint>
  <link name="camera_link"/>

  <joint name="camera_depth_joint" type="fixed">
    <parent link="camera_link"/>
    <child link="kinect_depth_frame"/>
    <origin xyz="0 0.025 0" rpy="-1.5708 0 -1.5708"/>
  </joint>
  <link name="kinect_depth_frame"/>

</robot>
```

UPDATED – Different from original guide

The depth frame link name is `kinect_depth_frame` to match the `output_frame` set in the `depth_to_scan` launch file.

Create package files:

```
nano ~/fyp_ws/src/qbot2_description/package.xml
```

Paste:

```
<?xml version="1.0"?>
<package format="3">
  <name>qbot2_description</name>
  <version>0.0.1</version>
  <description>QBot 2 URDF description</description>
  <maintainer email="test@test.com">kobuki</maintainer>
  <license>Apache-2.0</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <exec_depend>robot_state_publisher</exec_depend>
</package>
```

```
nano ~/fyp_ws/src/qbot2_description/CMakeLists.txt
```

Paste:

```
cmake_minimum_required(VERSION 3.8)
project(qbot2_description)
find_package(ament_cmake REQUIRED)
install(DIRECTORY urdf DESTINATION share/${PROJECT_NAME}/)
ament_package()
```

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

PHASE 8

Set Up Robot State Publisher

```
sudo apt install -y ros-humble-robot-state-publisher \  
ros-humble-joint-state-publisher
```

```
nano ~/fyp_ws/src/qbot2_bringup/launch/robot_state.launch.py
```

Paste:

```
import os  
from launch import LaunchDescription  
from launch_ros.actions import Node  
from ament_index_python.packages import get_package_share_directory  
  
def generate_launch_description():  
    urdf_file = os.path.join(  
        get_package_share_directory('qbot2_description'),  
        'urdf', 'qbot2.urdf')  
    with open(urdf_file, 'r') as f:  
        robot_description = f.read()  
    return LaunchDescription([  
        Node(  
            package='robot_state_publisher',  
            executable='robot_state_publisher',  
            parameters=[{'robot_description': robot_description}]  
        )  
    ])
```

```
cd ~/fyp_ws  
colcon build --symlink-install  
source ~/.bashrc
```

PHASE 9

Install SLAM Toolbox and Build the Map

UPDATED – Different from original guide

The original guide used RTAB-Map. SLAM Toolbox was used instead because it is simpler to configure with the fake laser scan from the Kinect. RTAB-Map can be used later for 3D mapping if needed.

Install SLAM Toolbox

```
sudo apt install -y ros-humble-slam-toolbox
```

Launch Everything for Mapping

Open 6 MobaXterm tabs. Keep all running simultaneously.

Tab 1 – Kobuki Driver

```
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash
ros2 launch kobuki_ros kobuki_node-launch.py
```

Tab 2 – Kinect Depth Node

```
cd ~/Ros2-KinectV1
source /opt/ros/humble/setup.bash
source install/setup.bash
sudo modprobe -r gspca_kinect
ros2 run ros2_kinect_depth depth_node
```

Tab 3 – Camera Info Fixer

```
source /opt/ros/humble/setup.bash
python3 ~/fix_kinect_camera_info.py
```

Tab 4 – Depth to LaserScan


```
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash
ros2 launch qbot2_bringup depth_to_scan.launch.py
```

Tab 5 – Robot State Publisher

```
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash
ros2 launch qbot2_bringup robot_state.launch.py
```

Tab 6 – SLAM Toolbox

```
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash
ros2 launch slam_toolbox online_async_launch.py \
  use_sim_time:=false
```

Verify All Topics are Running

In a new tab, check all required topics:

```
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash

ros2 topic list | grep -E "odom|scan|tf|map"

# You must see:
# /odom
# /scan
# /tf
# /tf_static
# /map
# /map_metadata
```

WARNING

If `/scan` does not appear, SLAM cannot build a map. Fix `/scan` first before starting SLAM. Check that Tab 2, Tab 3, and Tab 4 are all running.

Drive the Robot to Build the Map

```
sudo apt install -y ros-humble-teleop-twist-keyboard

# IMPORTANT: Kobuki uses /commands/velocity, not /cmd_vel
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -r cmd_vel:=/commands/velocity
```

UPDATED – Different from original guide

The Kobuki robot uses `/commands/velocity` topic, not the standard `/cmd_vel`. You must use the `-ros-args -r cmd_vel:=/commands/velocity` remapping or the robot will not move even though teleop appears to be running.

Key	Action	Tip
i	Move forward	Drive slowly for good map quality
,	Move backward	Use carefully
j	Turn left	Make slow smooth turns
l	Turn right	Avoid sharp fast spins
k	STOP	Press this first if unsure
q	Increase speed	Use sparingly
z	Decrease speed	Recommended for mapping

Watch the Map in RViz2

WINDOWS

Open a new MobaXterm SSH tab with X11 forwarding:

```
ssh -X kobuki@raspberrypi.local
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash
rviz2
```

In RViz2:

1. Set **Fixed Frame** to map
2. Click **Add** → By topic → `/map` → Map
3. Click **Add** → By topic → `/scan` → LaserScan
4. Click **Add** → TF

5. Drive the robot – the map should appear and grow

NOTE

The map stays empty until the robot moves. Drive slowly around the full lab area to build a complete map.

Save the Map

```
mkdir -p ~/maps
ros2 run nav2_map_server map_saver_cli -f ~/maps/lab_map

# Creates:
# ~/maps/lab_map.pgm
# ~/maps/lab_map.yaml
```

PHASE 10

Autonomous Navigation with Nav2

Install Nav2

```
sudo apt install -y ros-humble-navigation2 ros-humble-nav2-bringup
```

Create Nav2 Parameters File

```
mkdir -p ~/fyp_ws/src/qbot2_bringup/config  
nano ~/fyp_ws/src/qbot2_bringup/config/nav2_params.yaml
```

Paste:

```
amcl:  
  ros__parameters:  
    use_sim_time: false  
    alpha1: 0.2  
    alpha2: 0.2  
    alpha3: 0.2  
    alpha4: 0.2  
    base_frame_id: base_footprint  
    global_frame_id: map  
    laser_model_type: likelihood_field  
    max_beams: 60  
    scan_topic: /scan  
  
bt_navigator:  
  ros__parameters:  
    use_sim_time: false  
    global_frame: map  
    robot_base_frame: base_footprint  
  
controller_server:  
  ros__parameters:  
    use_sim_time: false  
    controller_frequency: 20.0  
    min_x_velocity_threshold: 0.001  
    min_y_velocity_threshold: 0.5  
    min_theta_velocity_threshold: 0.001  
  
local_costmap:  
  local_costmap:
```

```
ros__parameters:
  use_sim_time: false
  global_frame: odom
  robot_base_frame: base_footprint
  rolling_window: true
  width: 3
  height: 3
  resolution: 0.05

global_costmap:
  global_costmap:
    ros__parameters:
      use_sim_time: false
      global_frame: map
      robot_base_frame: base_footprint
      resolution: 0.05

planner_server:
  ros__parameters:
    use_sim_time: false
    planner_plugins: ['GridBased']
  GridBased:
    plugin: 'nav2_navfn_planner/NavfnPlanner'
    tolerance: 0.5
    use_astar: true
    allow_unknown: true
```

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

Launch Full Navigation Stack

Open 6 tabs (same as SLAM but replace SLAM Toolbox with Nav2):

```
# Tab 1 -- Kobuki:
ros2 launch kobuki_ros kobuki_node-launch.py

# Tab 2 -- Kinect depth:
cd ~/Ros2-KinectV1
source /opt/ros/humble/setup.bash && source install/setup.bash
sudo modprobe -r gspca_kinect
ros2 run ros2_kinect_depth depth_node

# Tab 3 -- Camera info fixer:
source /opt/ros/humble/setup.bash
```

```
python3 ~/fix_kinect_camera_info.py

# Tab 4 -- Depth to LaserScan:
source /opt/ros/humble/setup.bash && source ~/fyp_ws/install/setup.bash
ros2 launch qbot2_bringup depth_to_scan.launch.py

# Tab 5 -- Robot State Publisher:
source /opt/ros/humble/setup.bash && source ~/fyp_ws/install/setup.bash
ros2 launch qbot2_bringup robot_state.launch.py

# Tab 6 -- Nav2 with saved map:
source /opt/ros/humble/setup.bash && source ~/fyp_ws/install/setup.bash
ros2 launch nav2_bringup bringup_launch.py \
  map:=/home/kobuki/maps/lab_map.yaml \
  use_sim_time:=false \
  params_file:=/home/kobuki/fyp_ws/src/qbot2_bringup/config/nav2_params.yaml
```

Wait until Tab 6 shows:

```
[lifecycle_manager] Managed nodes are active
```

Send Navigation Goals with RViz2

WINDOWS

```
ssh -X kobuki@raspberrypi.local
source /opt/ros/humble/setup.bash
source ~/fyp_ws/install/setup.bash
rviz2
```

In RViz2:

1. Set Fixed Frame to map
2. Add /map, /scan, /plan, TF displays
3. Click **2D Pose Estimate** – click where robot is on map
4. Click **2D Nav Goal** – click target on map
5. Robot will plan path and navigate autonomously

Troubleshooting

Problem	Cause	Solution
<code>dpkg-print-architecture</code> command not found	Missing space in command	Use <code>dpkg -print-architecture</code> with a space
<code>ros-humble-openni2-launch</code> not found	Package does not exist in Humble	Use KinectV1-Ros2 package instead (Phase 5)
OpenNI2 shows “0 devices connected”	OpenNI2 does not support Kinect v1	Use KinectV1-Ros2 package instead
<code>LIBUSB_ERROR_BUSY</code>	Two nodes trying to access Kinect at same time	Run only one Kinect node. Use <code>sudo modprobe -r gspca_kinect</code> before starting
<code>/scan</code> not publishing	Camera info fixer not running	Start <code>python3 ~/fix_kinect_camera_info.py</code> in separate tab
Robot does not move with teleop	Wrong topic name	Use <code>-ros-args -r cmd_vel:=/commands/velocity</code>
<code>/map</code> not appearing	SLAM not running or <code>/scan</code> missing	Check <code>ros2 topic hz /scan</code> . Fix scan first then restart SLAM
Node name conflict warning	Duplicate SLAM nodes	Run <code>pkill -f slam_toolbox</code> then restart
Map is empty in RViz2	Robot has not moved	Drive robot slowly with teleop. Map builds as robot moves
Nav2 path not planning	Pose estimate not set	Click 2D Pose Estimate in RViz2 before sending goal

Quick Reference Commands

```
# Check all topics:
ros2 topic list | grep -E "odom|scan|tf|map|kinect"

# Check data rates:
ros2 topic hz /kinect/depth/image_raw    # expect 30Hz
ros2 topic hz /scan                       # expect 30Hz
ros2 topic hz /odom                       # expect data

# Emergency stop:
ros2 topic pub /commands/velocity \
  geometry_msgs/msg/Twist \
  '{linear: {x: 0.0}, angular: {z: 0.0}}' --once

# Save map:
ros2 run nav2_map_server map_saver_cli -f ~/maps/lab_map

# Rebuild workspace:
cd ~/fyp_ws && colcon build --symlink-install && source ~/.bashrc
```


14-Day Practical Plan

Day	Task	Done when
Day 1	Flash Ubuntu 22.04. Boot Pi. SSH from MobaXterm.	kobuki@pi:~\$ prompt appears
Day 2	Update Ubuntu. Install ROS 2 Humble with corrected commands.	Hello World messages appear
Day 3	Install Kobuki driver. Connect QBot 2. Test /odom.	/odom topic publishing
Day 4	Install KinectV1-Ros2. Test /kinect/depth/image_raw.	Depth topic at 30Hz
Day 5	Create camera info fixer. Install depth-to-laserscan. Test /scan.	/scan topic publishing
Day 6	Create URDF with kinect_depth_frame. Robot state publisher.	Build succeeds, TF publishing
Day 7	Install SLAM Toolbox. Launch all 6 nodes. Drive robot.	Map appears in RViz2
Day 8	Drive complete lab map. Save map files.	lab_map.pgm and lab_map.yaml saved
Day 9	Install Nav2. Configure params. Launch navigation.	Managed nodes are active
Day 10	Set Pose Estimate in RViz2. Send first navigation goal.	Robot reaches goal autonomously
Day 11	Test multiple goals. Verify obstacle avoidance.	Robot reliably reaches 5 different goals
Day 12	Record video demo of mapping and navigation.	Clear video saved
Day 13	Screenshots, results, note limitations.	Results documented
Day 14	Update final presentation with system diagram and results.	Slides complete

Follow every phase in order.

This guide reflects what actually worked on this hardware. Trust the UPDATED boxes.